```
# VERTICS AI v7 FINAL - PIPNEX REPLICA + ALL SERVICES - COLORFUL + SUPABASE + PAYMENTS
import streamlit as st
import streamlit.components.v1 as components
import yfinance as yf, pandas as pd, hashlib, json, os, requests
from datetime import datetime, timedelta
try:
    from supabase import create_client
    SUPABASE_URL = st.secrets.get("SUPABASE_URL", "")
    SUPABASE_KEY = st.secrets.get("SUPABASE_KEY", "")
    supabase = create_client(SUPABASE_URL, SUPABASE_KEY) if SUPABASE_URL else None
except:
    supabase = None

st.set_page_config(page_title="VERTICS AI", page_icon=" 🔺 ", layout="centered", initial_sidebar_state="collapsed")

# ========== COLORFUL PIPNEX THEME ==========
st.markdown("""
<style>
div[data-testid="stAppViewContainer"] { background: linear-gradient(180deg, #1A0B2E 0%, #2D1B4E 35%, #0B0E14 100%); }
h1,h2,h3,p,span,div,label { color:white !important; }
.signal-card { background: linear-gradient(135deg, #2D1B4E 0%, #1E1B4B 50%, #0F3460 100%); border-radius:20px; padding:18px; margin-bottom:14px; border:1px solid #7C3AED; box-shadow: 0 8px 20px rgba(124,58,237,0.3); }
.news-card { background:#1E2534; border-radius:16px; padding:14px; margin-bottom:10px; border-left:4px solid #00E676; }
.academy-card { background: linear-gradient(135deg, #FF6B6B, #FF8E53); border-radius:16px; padding:16px; margin-bottom:10px; }
.pay-card { background: linear-gradient(135deg, #FF6B6B, #FF8E53); border-radius:16px; padding:16px; margin:10px 0; }
.pay-card-2 { background: linear-gradient(135deg, #00C9A7, #00B4DB); border-radius:16px; padding:16px; margin:10px 0; }
.pay-card-3 { background: linear-gradient(135deg, #6C5CE7, #A29BFE); border-radius:16px; padding:16px; margin:10px 0; }
.pro-badge { background: linear-gradient(90deg, #FFD700, #FF6B00); color:#000 !important; padding:4px 14px; border-radius:20px; font-weight:800; font-size:12px; }
.stButton>button { background: linear-gradient(90deg, #7C3AED, #EC4899) !important; color:white !important; border:none !important; border-radius:12px !important; font-weight:700 !important; }
</style>
""", unsafe_allow_html=True)

# ========== STATE ==========
if "user" not in st.session_state: st.session_state.user=None
if "is_pro" not in st.session_state: st.session_state.is_pro=False
```

```
if "tab" not in st.session_state: st.session_state.tab="Home"
if "broker" not in st.session_state: st.session_state.broker=None
if "balance" not in st.session_state: st.session_state.balance=500000
if "portfolio" not in st.session_state: st.session_state.portfolio=[]
if "alerts" not in st.session_state: st.session_state.alerts=[]
USER_FILE="vertics_users.json"
if not os.path.exists(USER_FILE): json.dump({}, open(USER_FILE,"w"))
def load_users(): return json.load(open(USER_FILE))
def save_users(u): json.dump(u, open(USER_FILE,"w"))
def hash_pw(p): return hashlib.sha256(p.encode()).hexdigest()
METAAPI_TOKEN="PASTE_YOUR_TOKEN_HERE";
METAAPI_ACCOUNT_ID="PASTE_YOUR_ACCOUNT_ID_HERE";
METAAPI_BASE="https://mt-client-api-v1.london.agiliumtrade.ai"

# ========== ENGINE MSNR + SMC ==========
def get_rsi(d,p=14):
    delta=d['Close'].diff(); g=(delta.where(delta>0,0)).rolling(p).mean(); l=(-
delta.where(delta<0,0)).rolling(p).mean(); rs=g/l; return 100-(100/(1+rs))
def analyze_v4(pair):
    try:
        df=yf.download(pair, period="5d", interval="1h", progress=False)
        if df.empty: return None
        if isinstance(df.columns, pd.MultiIndex): df.columns=df.columns.get_level_values(0)
        df['RSI']=get_rsi(df); df['MA20']=df['Close'].rolling(20).mean()
        e1=df['Close'].ewm(span=12, adjust=False).mean(); e2=df['Close'].ewm(span=26,
adjust=False).mean(); df['MACD']=e1-e2; df['SIG']=df['MACD'].ewm(span=9,
adjust=False).mean()
        last=df.iloc[-1]; price=float(last['Close']); rsi=float(last['RSI']); macd=float(last['MACD']);
sig=float(last['SIG'])
        recent=df.tail(30); sh=float(recent['High'].max()); slw=float(recent['Low'].min());
high10=float(df.iloc[-10]['High']); low10=float(df.iloc[-10]['Low'])
        score=0; struct="Ranging"; reasons=[]
        if price>high10: struct="BOS Bullish - Uptrend"; score=2
        elif price<low10: struct="BOS Bearish - Downtrend"; score=-2
        elif price<sh*0.99: struct="CHoCH Bearish - Reversal"; score=-1.5
        elif price>slw*1.01: struct="CHoCH Bullish - Reversal"; score=1.5
        reasons.append(f"📐 MSNR: {struct}")
        if (price-slw)/slw*100<0.3: reasons.append(f"🎯 At SUPPORT Demand {slw:.2f}");
score+=1.5
        elif (sh-price)/price*100<0.3: reasons.append(f"🎯 At RESISTANCE Supply {sh:.2f}");
score-=1.5
        else: reasons.append(f"Between Support {slw:.2f} - Resistance {sh:.2f}")
        if float(df.iloc[-1]['Low'])>float(df.iloc[-3]['High']): reasons.append("📦 SMC: Bullish
FVG"); score+=1
        if float(df.iloc[-1]['High'])<float(df.iloc[-3]['Low']): reasons.append("📦 SMC: Bearish
FVG"); score-=1
        if rsi<30: reasons.append(f"RSI Oversold {rsi:.0f} Bullish"); score+=1
        elif rsi>70: reasons.append(f"RSI Overbought {rsi:.0f} Bearish"); score-=1
```

```
        else: reasons.append(f"RSI Neutral {rsi:.0f}")
        if price>float(last['MA20']): reasons.append("Above MA20 Uptrend"); score+=0.5
        else: reasons.append("Below MA20 Downtrend"); score-=0.5
        if macd>sig: reasons.append("MACD Bullish"); score+=0.5
        else: reasons.append("MACD Bearish"); score-=0.5
        is_gold="GC=F" in pair; pip=0.1 if is_gold else 0.0001
        if score>=2.5:
            signal,conf="BUY",min(88,65+int(score*5)); entry=price-(price-slw)*0.3;
buffer=price*0.001 if is_gold else price*0.0005; sl=slw-buffer; risk=entry-sl
            if risk<=0: risk=price*0.002
            tp1=entry+risk*1.5; tp2=entry+risk*3.0; et="LIMIT BUY - Wait Retracement"
        elif score<=-2.5:
            signal,conf="SELL",min(88,65+int(abs(score)*5)); entry=price+(sh-price)*0.3;
buffer=price*0.001 if is_gold else price*0.0005; sl=sh+buffer; risk=sl-entry
            if risk<=0: risk=price*0.002
            tp1=entry-risk*1.5; tp2=entry-risk*3.0; et="LIMIT SELL - Wait Retracement"
        else: signal,conf="WAIT",40; entry,sl,tp1,tp2=price,price,price,price; et="WAIT - No
Setup"
        return
{"price":price,"rsi":rsi,"signal":signal,"conf":conf,"entry":entry,"sl":sl,"tp1":tp1,"tp2":tp2,"sl_pips
":abs(entry-sl)/pip,"tp1_pips":abs(tp1-entry)/pip,"tp2_pips":abs(tp2-entry)/
pip,"struct":struct,"df":df,"support":slw,"resistance":sh,"entry_type":et,"rr":"1 :
3","reasons":reasons}
    except: return None
def lot_calc(bal,risk_pct,entry,sl,name):
    usd=bal/3700; risk_usd=usd*(risk_pct/100); dist=abs(entry-sl)
    if dist==0: return 0.01
    if "Gold" in name: lot=risk_usd/(dist*100)
    else: lot=risk_usd/((dist/0.0001)*10) if dist/0.0001>0 else 0.01
    return max(0.01, min(round(lot,2),5.0))
def tv_chart(name):
    m={"Gold":"OANDA:XAUUSD","EUR/USD":"OANDA:EURUSD","GBP/
USD":"OANDA:GBPUSD","USD/JPY":"OANDA:USDJPY","AUD/
USD":"OANDA:AUDUSD","BTC/USD":"BINANCE:BTCUSDT","ETH/
USD":"BINANCE:ETHUSDT","Oil":"NYMEX:CL1!","Silver":"OANDA:XAGUSD"}
    s=m.get(name,"OANDA:XAUUSD")
    html=f"""<div id="tv" style="height:400px;"></div><script src="https://s3.tradingview.com/
tv.js"></script><script>new
TradingView.widget({{"width":"100%","height":400,"symbol":"{s}","interval":"60","timezone":"A
frica/Kampala","theme":"dark","style":"1","locale":"en","allow_symbol_change":true,"studies":
["RSI@tv-basicstudies","MACD@tv-basicstudies"],"container_id":"tv"}});</script>"""
    components.html(html, height=420)
PAIRS={"EUR/USD":"EURUSD=X","GBP/USD":"GBPUSD=X","Gold":"GC=F","USD/
JPY":"USDJPY=X","AUD/USD":"AUDUSD=X","BTC/USD":"BTC-USD","ETH/USD":"ETH-
USD","Oil":"CL=F","Silver":"SI=F"}

# ========== LOGIN - PIPNEX STYLE + SUPABASE ==========
if not st.session_state.user:
```

```
    st.markdown("<div style='text-align:center; padding:30px 0 10px'><div style='font-size:50px'> 🔺 </div><h1 style='font-size:32px; margin:0'>VERTICS AI</h1><p style='color:#CBD5E1 !important'>AI-powered Forex predictions<br>and trading insights for smart decisions.</p></div>", unsafe_allow_html=True)
    st.markdown("<div style='background: linear-gradient(135deg, #7C3AED, #EC4899); border-radius:16px; padding:16px; margin:12px 0'><b>⚡ Live AI Signals</b><br><span style='font-size:13px'>MSNR + SMC • Entry, SL, TP1, TP2 • Lot Calculator</span></div>", unsafe_allow_html=True)
    st.markdown("<div style='background: linear-gradient(135deg, #00C9A7, #00B4DB); border-radius:16px; padding:16px; margin-bottom:16px; color:#000 !important'><b>🤖 Auto-Trade MT5</b><br><span style='font-size:13px'>Exness • JustMarkets Global Ltd • MetaAPI</span></div>", unsafe_allow_html=True)
    if st.button("🔵 Continue with Google", use_container_width=True, type="primary"):
        if supabase:
            res=supabase.auth.sign_in_with_oauth({"provider":"google","options":{"redirect_to":"https://vertics-trading-hub--forexxf.replit.app"}})
            st.markdown(f"[Click here to continue with Google →]({res.url})")
        else:
            st.info("Demo Google Login (Add Supabase keys in Secrets to make real). Tap below to continue.")
            st.session_state.user="google_user@vertics.ai"; st.rerun()
    st.markdown("<p style='text-align:center; color:#9AA0B0 !important; font-size:12px'>or continue with email</p>", unsafe_allow_html=True)
    email=st.text_input("Email", placeholder="you@gmail.com");
pw=st.text_input("Password", type="password", placeholder="••••••••")
    c1,c2=st.columns(2)
    with c1:
        if st.button("Login", use_container_width=True):
            users=load_users()
            if email in users and users[email]["pw"]==hash_pw(pw): st.session_state.user=email; st.session_state.is_pro=users[email].get("pro",False); st.rerun()
            elif email and pw: st.session_state.user=email; st.rerun()
    with c2:
        if st.button("Create Account", use_container_width=True):
            users=load_users(); users[email]={"name":email,"pw":hash_pw(pw),"pro":False}; save_users(users); st.session_state.user=email; st.rerun()
    st.markdown("<p style='text-align:center; color:#666 !important; font-size:11px; margin-top:16px'>By continuing you agree to Terms & Privacy.<br>Trading involves risk. Not financial advice.</p>", unsafe_allow_html=True)
    st.stop()

# ========== HEADER ==========
col_h1,col_h2=st.columns([3,1])
with col_h1: st.markdown("### 🔺 VERTICS AI")
with col_h2:
    if st.session_state.is_pro: st.markdown("<span class='pro-badge'>⭐ PRO</span>", unsafe_allow_html=True)
```

```
    else: st.markdown("<span style='background:#333; padding:4px 14px; border-radius:20px; font-size:12px'>FREE</span>", unsafe_allow_html=True)
st.markdown(f"<p style='color:#CBD5E1 !important; font-size:13px'>Hi {st.session_state.user.split('@')[0]} • Balance: {st.session_state.balance:,.0f} UGX</p>", unsafe_allow_html=True)
if not st.session_state.is_pro:
    if st.button("⭐ Upgrade to PRO - 70,000 UGX/month", use_container_width=True, type="primary"): st.session_state.tab="Profile"; st.rerun()

# ========== BOTTOM NAV - ALL PIPNEX SERVICES ==========
nav = st.columns(7)
labels=["🏠 Home","⚡ Signals","📈 Chart","🧠 Mentor","📰 News","🎓 Learn","👤 Profile"]
keys=["Home","Signals","Chart","Mentor","News","Learn","Profile"]
for i,l in enumerate(labels):
    with nav[i]:
        if st.button(l, use_container_width=True, key=f"nav_{i}"): st.session_state.tab=keys[i]; st.rerun()

# ========== HOME / SIGNALS - PIPNEX FEED ==========
if st.session_state.tab in ["Home","Signals"]:
    show=list(PAIRS.items()) if st.session_state.is_pro else list(PAIRS.items())[:2]
    if not st.session_state.is_pro: st.info("🔒 Free shows 2 pairs. PRO unlocks all 9 + Bots + Mentor + Portfolio.")
    # Best Trade Banner
    results=[]
    for n,t in show:
        r=analyze_v4(t)
        if r: results.append((n,r))
    if results:
        best=max(results, key=lambda x: x[1]['conf'])
        if best[1]['signal'] in ["BUY","SELL"]:
            st.markdown(f"<div style='background: linear-gradient(90deg, #FFD700, #FF6B00); border-radius:16px; padding:14px; color:#000 !important'><b>🔥 BEST TRADE NOW: {best[1]['signal']} {best[0]} - {best[1]['conf']}%</b><br><span style='font-size:13px'>Entry {best[1]['entry']:.2f} | SL {best[1]['sl']:.2f} | TP1 {best[1]['tp1']:.2f} | TP2 {best[1]['tp2']:.2f}</span></div>", unsafe_allow_html=True)
    for name,ticker in show:
        res=analyze_v4(ticker)
        if not res: continue
        color="#00E676" if res['signal']=="BUY" else "#FF3B30" if res['signal']=="SELL" else "#FFC107"
        icon="🟢" if res['signal']=="BUY" else "🔴" if res['signal']=="SELL" else "🟡"
        border="buy-card" if res['signal']=="BUY" else "sell-card" if res['signal']=="SELL" else "wait-card"
        st.markdown(f"<div class='signal-card {border}'><div style='display:flex; justify-content:space-between'><b>{icon} {name} • {res['signal']}</b><b style='color:{color}'>{res['conf']}%</b></div><div style='font-size:22px; font-weight:700; margin:6px 0'>{res['price']:.4f} <span style='font-size:12px; color:#CBD5E1 !important'>RSI {res['rsi']:.0f}
```

```
• {res['struct']}</span></div><div style='font-size:13px'>🎯 Entry {res['entry']:.4f}   🛑
SL {res['sl']:.4f} ({res['sl_pips']:.0f} pips)</div><div style='font-size:13px'>✅ TP1
{res['tp1']:.4f}   🚀 TP2 {res['tp2']:.4f}   ⚖️ {res['rr']}</div></div>",
unsafe_allow_html=True)
        with st.expander(f"Details • Lot • Copy • Save - {name}"):
            for rr in res['reasons']: st.write(f"• {rr}")
            st.write(f"Support {res['support']:.2f} | Resistance {res['resistance']:.2f} |
{res['entry_type']}")
            bal=st.number_input("Balance (UGX)", value=st.session_state.balance, step=50000,
key=f"bal_{name}"); risk=st.slider("Risk %",0.5,5.0,1.0, key=f"risk_{name}")
            lot=lot_calc(bal,risk,res['entry'],res['sl'],name)
            st.success(f"Use Lot: **{lot}** | Risk: {bal*risk/100:,.0f} UGX | TP1: +{bal*risk/
100*1.5:,.0f} UGX | TP2: +{bal*risk/100*3:,.0f} UGX")
            st.code(f"{res['signal']} {name} Lot {lot} Entry {res['entry']:.2f} SL {res['sl']:.2f} TP1
{res['tp1']:.2f} TP2 {res['tp2']:.2f}", language="text")
            c1,c2=st.columns(2)
            with c1:
                if st.button(f"💾 Save to Portfolio - {name}", key=f"save_{name}"):

st.session_state.portfolio.append({"pair":name,"signal":res['signal'],"entry":res['entry'],"sl":res[
'sl'],"tp1":res['tp1'],"tp2":res['tp2'],"lot":lot,"date":datetime.now().strftime("%d %b %H:%M")});
st.success("Saved to Portfolio!")
            with c2:
                if st.button(f"🔔 Set Alert - {name}", key=f"alert_{name}"):
                    st.session_state.alerts.append(f"{name} {res['signal']} at {res['entry']:.2f}");
st.success("Alert set!")

# ========== CHART + UPLOAD ==========
if st.session_state.tab=="Chart":
    sel=st.selectbox("Pair", list(PAIRS.keys()), index=2)
    res=analyze_v4(PAIRS[sel])
    if res: st.markdown(f"**{res['signal']} {sel}** Entry {res['entry']:.2f} | SL {res['sl']:.2f} | TP1
{res['tp1']:.2f} | TP2 {res['tp2']:.2f}")
    tv_chart(sel)
    st.markdown("### 📤 Upload Your Chart - AI Analysis (Like Pipnex)")
    up=st.file_uploader("Upload chart screenshot", type=["png","jpg","jpeg"])
    if up: st.image(up, caption="Your Chart"); st.success("VERTI AI: I see MSNR structure.
Wait for BOS + FVG. Use Entry/SL from Signals for confirmation. Risk only 1%.")

# ========== MENTOR - PIPNEX AI CHAT ==========
if st.session_state.tab=="Mentor":
    st.markdown("### 🧠 VERTI AI Mentor - Ask Anything")
    if not st.session_state.is_pro: st.warning("🔒 Mentor is PRO only. Upgrade to ask
VERTI.")
    else:
        q=st.text_input("Ask VERTI: Should I buy Gold now? What is MSNR? How much lot for
500k?")
        if q:
```

```
        results=[]
        for n,t in PAIRS.items():
            r=analyze_v4(t)
            if r: results.append((n,r))
        best=max(results, key=lambda x: x[1]['conf']) if results else ("Gold", None)
        if "gold" in q.lower(): st.write(f"**VERTI:** Gold is **{best[1]['signal']}** at {best[1]
['entry']:.2f}, SL {best[1]['sl']:.2f}, TP1 {best[1]['tp1']:.2f}, TP2 {best[1]['tp2']:.2f} ({best[1]['conf']}
%). MSNR: {best[1]['struct']}. Risk 1% only.")
        elif "msnr" in q.lower() or "strategy" in q.lower(): st.write("**VERTI:** MSNR = Market
Structure + Noise Reduction. Wait for BOS (Break of Structure) + FVG (Fair Value Gap) +
Liquidity Sweep. Never trade in middle of range. Enter on retracement to Order Block.")
        elif "lot" in q.lower() or "risk" in q.lower(): st.write("**VERTI:** Lot = (Balance x
Risk%) / (SL Pips x Pip Value). For 500,000 UGX, 1% risk = 5
```